

Agente de Batalha Naval em OCaml

Este projeto implementa um agente autónomo para jogar Batalha Naval, desenvolvido em OCaml. O agente gera a sua própria frota e implementa uma estratégia de ataque determinística e otimizada baseada em paridade.

1. Estruturas de Dados

O estado do jogo é mantido através de **tipos algébricos** e **registos**, garantindo clareza na gestão dos dois tabuleiros.

Tipos Principais

- `tipo_barco` : Variante que define os 5 tipos de navios PortaAvioes, Destroyer, Fragata, Torpedeiro, Submarino.
- `coord` : Um par de inteiros `(linha, coluna)` representando uma posição.

Representação dos Tabuleiros

Os tabuleiros são representados por Matrizes Bidimensionais (*Arrays de Arrays*) de tamanho $N \times N$ (padrão 8x8).

- `tabuleiro_defesa` (`celula_defesa array array`): Regista onde estão os nossos barcos e onde o adversário acertou.
 - **Estados possíveis:** `Mar`, `Navio of tipo`, `Atingido of tipo`, `ErroAdversario`.
- `tabuleiro_ataque` (`celula_ataque array array`): O "mapa de conhecimento" sobre o adversário.
 - **Estados possíveis:** `Desconhecido`, `Agua`, `Fogo` (acerto não afundado), `Afundado`.

Gestão da Frota e Alvos

- `minha_frota` : Uma lista de registos `info_barco`, onde cada barco sabe as suas coordenadas e quantas vidas (células intactas) lhe restam. Isto permite responder "afundado" instantaneamente.

- `pilha_alvos` : Uma lista de coordenadas (`int * int`) que funciona como uma pilha (*Stack*). Armazena os tiros prioritários quando o agente entra em modo "Caça" ou "Destruição".

2. Compilação e Execução

O projeto requer o compilador OCaml instalado (via `opam`).

Como Compilar

Para gerar o executável `agenteIA.exe` :

```
opam exec -- ocaml-opt -o agenteIA.exe agenteIA.ml
```

Como Executar

Basta correr o executável gerado `./agenteIA.exe`. O agente ficará à espera de comandos.

3. Estratégia de IA Implementada

O agente utiliza uma estratégia dividida em três fases para maximizar a eficiência dos tiros.

A. Modo de Busca (Estratégia de Xadrez / Paridade)

Enquanto não há navios detetados, o agente não dispara aleatoriamente. Utiliza uma **Estratégia de Paridade**:

1. O tabuleiro é tratado como um tabuleiro de xadrez (casas pretas e brancas).
2. O agente dispara apenas nas casas "pretas" (onde a soma `linha + coluna` é par).

Justificação: O menor navio (Torpedeiro, 2 células) ocupa obrigatoriamente pelo menos uma casa preta. Isto reduz o espaço de busca em 50%, garantindo matematicamente que nenhum navio consegue esconder-se.

B. Modo "Caça"

Assim que o agente recebe a resposta `tiro <barco>` (acerto):

1. O estado da célula passa a `Fogo`.
2. O agente calcula as coordenadas vizinhas válidas (**Cima, Baixo, Esquerda, Direita**).
3. Estas coordenadas são adicionadas à `pilha_alvos` para serem processadas imediatamente nos turnos seguintes.

C. Modo "Destruição"

Se o agente conseguir **dois acertos consecutivos** no mesmo navio (dois **Fogo** adjacentes):

1. O algoritmo infere a orientação do barco (**Horizontal** ou **Vertical**).
2. A **pilha_alvos** é filtrada para conter apenas as coordenadas que seguem essa linha.
3. Isto evita disparar para os lados desnecessariamente quando a direção do navio já é conhecida.

Otimizações Adicionais

- **Área de Exclusão (Afundado):** Quando um navio é afundado, o agente usa um algoritmo de busca (**BFS**) para identificar todas as células do navio e marca automaticamente todas as células vizinhas como **Água**. Isto explora a regra de que "os barcos não se podem tocar", poupando tiros.
- **Memória de Focos:** Se o agente estiver a atacar um barco e, por sorte, acertar noutro barco adjacente, ele não "esquece" o segundo barco. Após afundar o primeiro, ele reativa o foco no segundo barco.